

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W



LEARNER GUIDE

For

Microsoft Azure Fundamentals (AZ-900)

TGS Ref No: TGS-2023036449

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
1	8 July 2026	First Azure AZ-900 courseware build aligned to all 16 hands-on labs.	Tertiary Infotech Academy Pte Ltd

Course Information

Welcome! This Step-by-step lab guide takes you through the hands-on labs in the WSQ Microsoft Azure Fundamentals (AZ-900) course. Learners use Azure Portal, Azure Cloud Shell and Azure web tools to practise the concepts required for AZ-900.

Note: Most labs use an Azure free account and the shared resource group **rg-az900-labs**. Always complete the cleanup instructions to avoid unnecessary cloud charges.

Learning Outcomes

By the end of this course, learners will be able to:

- Describe cloud concepts, Azure regions, availability zones and Azure management tools.
- Create and inspect common Azure compute, networking, storage and database services.
- Use Microsoft Entra ID and RBAC to explain identity and access control fundamentals.
- Apply basic governance with tags, locks, Azure Policy, budgets and monitoring services.

Skills Framework Alignment

Course Reference: TGS-2023036449 | Certification Alignment: Microsoft Azure Fundamentals (AZ-900)

Course Outline

Domain 1: Describe cloud concepts (25-30%)

- Azure portal tour
- Azure Cloud Shell
- Regions, region pairs and availability zones

Domain 2: Describe Azure architecture and services (35-40%)

- Virtual machines
- App Service
- Container Instances
- Virtual networks
- Storage, blobs and Azure SQL
- Microsoft Entra ID

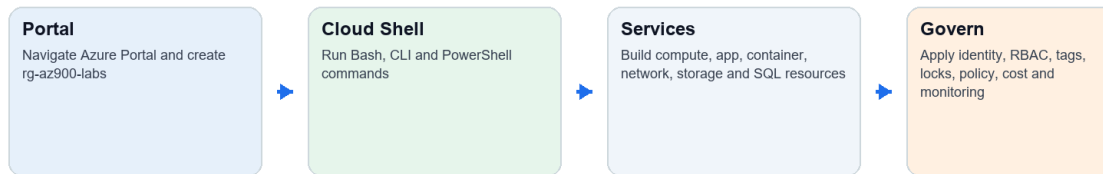
Domain 3: Describe Azure management and governance (30-35%)

- RBAC

- Tags and locks
- Azure Policy
- Cost management
- Advisor, Service Health and Azure Monitor

Diagram - Two-Day Azure Lab Flow

Two-Day Azure Lab Flow

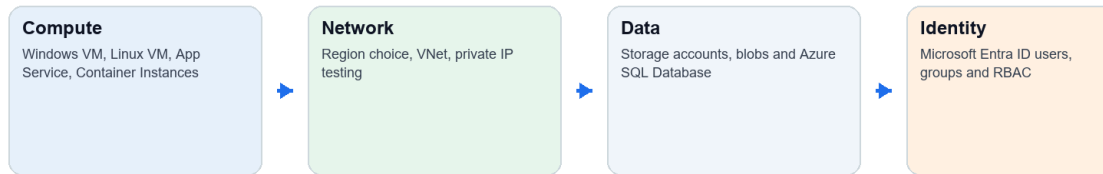


All diagrams are aligned to the 16 Azure hands-on labs and reused across PPT, Learner Guide and Lesson F

Use this flow to orient the labs: portal and shell basics first, then services, then governance.

Diagram - Azure Services Practised

Azure Services Practised



All diagrams are aligned to the 16 Azure hands-on labs and reused across PPT, Learner Guide and Lesson F

The 16 labs cover compute, networking, data, identity and governance services.

Diagram - AZ-900 Domain Weight Matrix

AZ-900 Domain Weight Matrix

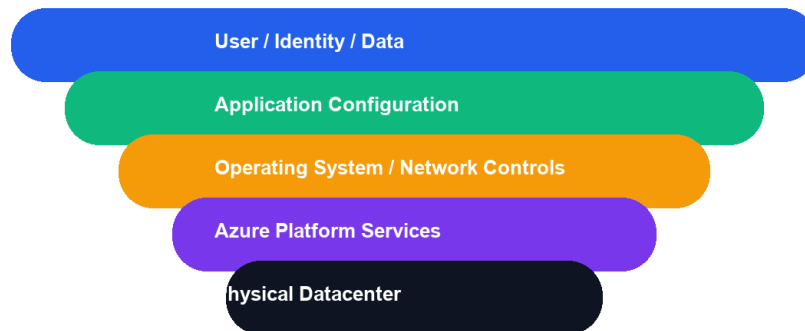
Domain	Exam Weight	Labs	Practice Priority
Cloud Concepts	25-30%	1-3	Moderate
Architecture & Services	35-40%	4-11	High
Management & Governance	30-35%	12-16	High

Use this matrix to prioritise revision and lab recap time before assessment.

Use the matrix to prioritise recap time across the three AZ-900 exam domains.

Diagram - Azure Shared Responsibility Layers

Azure Shared Responsibility Layers



AZ-900 labs show which controls learners configure and which controls Azure operates.

This layered model clarifies which cloud responsibilities learners configure in the labs.

Hands-On Lab Guide

Lab 1 - Tour the Azure Portal and Create Your First Resource Group

In this lab you take a guided tour of the Azure portal - the web-based console you will use throughout this course - and create the resource group **rg-az900-labs** that hosts every resource you build over the next two days. This maps to the AZ-900 skill areas **Describe cloud concepts** (getting started with a cloud provider) and **Describe Azure management and governance** (portal, resource groups, tags). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 1 | Domain 1 - Describe cloud concepts | Source: [labs/lab-01-azure-portal-tour.md](#)

Step 1 - Sign in to the Azure portal

1. Open a browser and go to **https://portal.azure.com**.
2. Sign in with the Microsoft account tied to your Azure free account. If you do not have one yet, create it first at **https://azure.microsoft.com/free** (requires a credit card for identity verification - you are **not** charged; the free account includes USD 200 credit for 30 days plus always-free services).
3. If prompted, complete multi-factor authentication and choose **Yes** on "Stay signed in?" for the duration of the class.

The portal Home page loads with **Azure services** icons at the top and **Recent resources** below.

Step 2 - Explore the portal menu and global navigation

4. Click the **hamburger icon** (, top-left) to open the **portal menu**. Note the entries: **Home**, **Dashboard**, **All services**, **All resources**, **Resource groups**, plus favorited services with stars.
5. Click **All services** and browse the categories on the left (Compute, Networking, Storage, Databases, etc.). This is the full Azure service catalogue - 200+ services.
6. Hover over **Virtual machines** and click the **star** icon to add it to your favourites in the portal menu.
7. Click **Home** in the portal menu to return.

Everything you can do in Azure is reachable from this one menu - the portal is just a GUI over the same Azure Resource Manager (ARM) API that the CLI and PowerShell use.

Step 3 - Use the global search bar

8. Click the **search bar** at the top of the portal (or press **/** or **Ctrl+/** as a shortcut).
9. Type `resource groups` - note results are grouped into **Services**, **Resources**, **Marketplace** and **Documentation**.
10. Type `virtual machines` and observe the same grouping, then press **Esc** to close.

The search bar is the fastest way to navigate - during labs, searching a service name beats clicking through menus.

Step 4 - Explore the top toolbar icons

Locate these icons at the top-right of the portal and click each briefly:

|  | **Icon** | **Name** | What it does |

| ---|---|---|

|  | **Cloud Shell** | Opens a browser-based Bash/PowerShell terminal (Lab 2) - close it for now |

|  | **Notifications** | Shows deployment progress and alerts - currently empty |

|  | **Settings** | Portal appearance, language, startup view |

|  | **Help + support** | Documentation and support tickets |

Step 5 - Customise portal settings

11. Click the **gear icon** (Settings) at the top-right.
12. Under **Appearance + startup views**:
 - **Choose a theme**: try a dark theme, then set it back to your preference.
 - **Startup page**: options are **Home** or **Dashboard** - leave it on **Home**.
13. Note the **Language + region** tab (portal display language) and the **Signing out + notifications** tab (inactivity sign-out timeout).
14. Click **Apply** if you changed anything, then close the Settings pane.

Step 6 - View and edit your dashboard, and pin a service

15. In the portal menu (☰), click **Dashboard**. You see the default private dashboard with tiles such as **All resources** and **Quickstarts + tutorials**.
16. Click **Edit** on the dashboard toolbar - a **Tile Gallery** appears. Drag any tile (e.g. **Clock**) onto the canvas, then click **Save**.
17. Now pin a service: search for `Virtual machines` in the global search, open it, and click the **pin icon** (📌) at the top-right of the Virtual machines blade. Choose your dashboard and click **Pin**.
18. Return to **Dashboard** and confirm the **Virtual machines** tile appears.

Dashboards are per-user, customisable views - teams often build shared dashboards for monitoring.

Step 7 - Create the resource group `rg-az900-labs`

19. In the global search bar, type `resource groups` and open **Resource groups**.
20. Click **+ Create**.
21. On the **Basics** tab enter:

- Subscription: your free-account subscription (e.g. `Azure subscription 1` or `Free Trial`)
- Resource group: `rg-az900-labs`
- Region: `Southeast Asia`

22. Click **Next: Tags**.

A resource group is a logical container for resources that share a lifecycle - deleting the group deletes everything inside it. The region here only stores the group's metadata; resources inside it can live in any region.

Step 8 - Add a tag and create

23. On the **Tags** tab enter:

- Name: `course`
- Value: `az900`

24. Click **Review + create** - validation runs and shows **Validation passed**.
25. Click **Create**. Creation is instant (resource groups contain no billable resources by themselves).
26. Click the **notifications bell** - you see the "Created resource group" notification.

Tags are name/value metadata used for cost reporting, ownership and automation - a core AZ-900 governance topic.

Step 9 - Verify the resource group

27. Open **Resource groups** from the portal menu and confirm `rg-az900-labs` is listed with location **Southeast Asia**.
28. Click `rg-az900-labs` to open its **Overview** blade - it shows **No resources to display** for now.
29. On the left menu of the resource group, note the blades you will use later: **Access control (IAM)**, **Tags**, **Deployments**, **Locks**.
30. Click **Tags** on the left and confirm `course : az900` is present.

Clean up

Nothing to delete. **Keep `rg-az900-labs`** - it is reused by every lab in this course and is only deleted at the end of the course (Lab 16). A resource group with no resources in it costs nothing.

Optional: if you added extra dashboard tiles you don't want, open **Dashboard** → **Edit**, remove them and **Save**.

What you learned

- How to navigate the Azure portal: portal menu, global search, All services, notifications and Cloud Shell icons.
- How to customise the portal experience (theme, startup page) and pin services/tiles to a dashboard.
- What a resource group is - a logical, lifecycle-based container for Azure resources - and that its region only stores metadata.
- How to apply tags (`course=az900`) for organisation and cost governance.
- That the portal, CLI and PowerShell are all front ends to the same Azure Resource Manager control plane.

Lab 2 - Get Started with Azure Cloud Shell (CLI and PowerShell)

In this lab you open Azure Cloud Shell - a browser-based terminal with the Azure CLI and Azure PowerShell pre-installed and pre-authenticated - and use it to inspect your subscription, manage the `rg-az900-labs` resource group, and edit files with the built-in editor. This maps to the AZ-900 skill area **Describe Azure management and governance** (tools for managing Azure: Azure portal, Azure CLI, Azure PowerShell, Cloud Shell). Run in Azure Cloud Shell - <https://shell.azure.com>

Note: Day 1 | Domain 1 - Describe cloud concepts | Source: `labs/lab-02-cloud-shell-cli.md`

Step 1 - Open Cloud Shell and choose Bash

31. Go to `https://shell.azure.com` (or in the portal, click the **Cloud Shell icon** `>_`` in the top toolbar).
32. Sign in with your Azure account if prompted.
33. When asked to choose a shell, select **Bash**.

Cloud Shell runs a small Linux container in Azure - you never install anything locally, and you are already signed in, so no `az login` is needed.

Step 2 - Complete the first-run storage setup

On first launch Cloud Shell asks how to persist your files:

34. If you see **Getting started** with a **No storage account required** option (ephemeral session), you can pick it for quick tasks - but for this course choose **Mount storage account** so your files survive between sessions.
35. Select your subscription and click **Apply** (or use **Advanced settings** to choose the region `Southeast Asia` and an existing resource group).
36. Wait for the prompt to appear, e.g. `yourname@Azure:~$``.

The mounted storage account holds a small file share (`clouddrive`) mapped into your home directory. The ephemeral "no storage" option starts faster but wipes files when the session ends.

Step 3 - Inspect your account with the Azure CLI

```
az account show
```

Displays the subscription Cloud Shell is signed into as JSON - check the `name`, `id` and `user` fields to confirm you are in the right subscription.

```
az account show --query name -o tsv
```

The `--query` flag uses JMESPath to extract a single field, and `-o tsv` strips the quotes - a pattern you will use constantly for scripting.

Step 4 - List resource groups

```
az group list -o table
```

Shows the resource groups in your subscription in a readable table - you should see `rg-az900-labs` from Lab 1 with location `southeastasia`.

The `-o table` output format is the most readable for humans; the default `json` is best for scripts.

Step 5 - Create the resource group with the CLI (idempotent)

```
az group create --name rg-az900-labs --location southeastasia --tags course=az900
```

Returns the group's JSON with `"provisioningState": "Succeeded"` - because ARM deployments are idempotent, re-running this on the existing group simply succeeds without error (and ensures the `course=az900` tag is set). If you skipped Lab 1, this creates the group fresh.

```
az group show --name rg-az900-labs -o table
```

Confirms the group exists and shows its location and status.

Step 6 - (Optional) Try interactive mode

```
az interactive
```

Launches an interactive CLI experience with auto-completion, command descriptions and examples as you type. It may take a minute to install on first run. Type `exit` to leave it and return to Bash.

Step 7 - Switch to PowerShell and run an Az cmdlet

37. In the Cloud Shell toolbar, use the shell drop-down (top-left, showing `**Bash**`) and select `**PowerShell**` (click `**Confirm**`). The prompt changes to ``PS /home/yourname>``.

```
Get-AzResourceGroup
```

Lists the same resource groups using the Azure PowerShell `Az` module - note `rg-az900-labs` appears with `Location: southeastasia`. Azure CLI and Azure PowerShell are two clients for the same ARM API; teams pick whichever fits their scripting culture.

```
Get-AzResourceGroup -Name rg-az900-labs | Format-List
```

Shows the full detail of one group, including the `course=az900` tag.

38. Switch back to `**Bash**` using the same drop-down for the remaining steps.

Step 8 - Use the built-in editor

```
mkdir -p ~/az900 && cd ~/az900
code notes.txt
```

`code . / code <file>` opens the built-in Cloud Shell editor (you can also click the **curly-brace icon** `{ }` in the toolbar). Type a line such as `AZ-900 Lab 2 - Cloud Shell works!`, then save with **Ctrl+S** and close the editor with **Ctrl+Q** (or the `...` menu → **Save / Close editor**).

```
cat notes.txt
```

Prints the file contents back - proof the file was written to your Cloud Shell home directory (persisted if you mounted storage in Step 2).

Step 9 - Upload and download files

39. In the Cloud Shell toolbar, click the `**Manage files**` icon (folder/up-down arrows) and choose `**Upload**`. Pick any small file from your computer - it lands in your Cloud Shell home directory (``ls ~`` to verify).

40. Choose **Manage files** → **Download**, enter the path ``az900/notes.txt``, and click **Download** - the browser downloads your file.

This is the simplest way to move scripts and templates between your laptop and Cloud Shell.

Clean up

Keep everything:

- ``rg-az900-labs`` is reused by all later labs - do not delete it (it is removed at the end of the course, Lab 16).
- The Cloud Shell storage account is tiny (a few GB of Standard LRS file share) and costs only cents per month; keep it for the rest of the course. If you ever want to remove it after the course, delete the resource group named like ``cloud-shell-storage-southeastasia``.

Remove only the scratch file if you like:

```
rm -rf ~/az900
```

What you learned

- Cloud Shell gives you a pre-authenticated Bash or PowerShell terminal in the browser - no local install, with optional persistent storage (or an ephemeral no-storage session).
- Core Azure CLI patterns: ``az account show``, ``az group list/create/show``, ``--query`` with JMESPath and ``-o table|tsv|json`` output formats.
- ARM operations are idempotent - re-running ``az group create`` on an existing group succeeds safely.
- Azure PowerShell (``Get-AzResourceGroup``) and the Azure CLI are equivalent management tools over the same Azure Resource Manager API.
- The built-in editor (``code``) and upload/download buttons make Cloud Shell a complete lightweight management workstation.

Lab 3 - Explore Regions, Region Pairs and Availability Zones

In this lab you explore Azure's global infrastructure - regions, region pairs and availability zones - using the CLI, Microsoft's interactive datacenter map, and a latency test tool, and you reason about how each construct delivers high availability versus disaster recovery. This maps to the AZ-900 skill areas **Describe cloud concepts** (high availability, disaster recovery, reliability) and **Describe Azure architecture and services** (regions, region pairs, availability zones). Run in Azure Cloud Shell - <https://shell.azure.com>

Note: Day 1 | Domain 1 - Describe cloud concepts | Source: [labs/lab-03-regions-availability-zones.md](#)

Step 1 - List all Azure regions with the CLI

```
az account list-locations -o table
```

Lists every region your subscription can see - 60+ rows with **DisplayName**, **Name** (the CLI-friendly id like **southeastasia**) and **RegionalDisplayName**. Note the **Name** column values: these are what you pass to **--location** in every **az** command.

```
az account list-locations --query "[?metadata.regionCategory=='Recommended'].{Region:displayName, Name:name, Pair:metadata.pairedRegion[0].name}" -o table
```

Filters to Microsoft's "Recommended" regions and shows each region's **paired region** - look at the **Pair** column for the row **Southeast Asia**.

Step 2 - Explore the global infrastructure map

41. Open **https://datacenters.microsoft.com/globe/explore** in a new browser tab.
42. Rotate the 3D globe to Asia and click the **Southeast Asia** region (Singapore). Read the region facts panel - launch year, availability zone support, sustainability info.
43. Zoom out and observe how regions cluster into **geographies** (e.g. Asia Pacific, Europe, United States) and how undersea network cables connect them.
44. Also open **https://azure.microsoft.com/en-us/explore/global-infrastructure/geographies/** and locate **Singapore** under Asia Pacific - confirm Southeast Asia is listed with availability zones.

A **geography** keeps data residency and compliance boundaries (important for regulated industries); a **region** is a set of datacenters within a latency-defined perimeter.

Step 3 - Identify Southeast Asia's paired region

45. From the CLI output in Step 1 (or the geographies page), confirm that **Southeast Asia** (Singapore) is paired with **East Asia** (Hong Kong).
46. Verify with a targeted query:

```
az account list-locations --query "[?name=='southeastasia'].{Region:displayName, PairedWith:metadata.pairedRegion[0].name}" -o table
```

Expected output shows **PairedWith: eastasia**. Region pairs are at least 300 miles apart in the same geography (where possible); Microsoft sequences platform updates across the pair and prioritises one region's recovery in a broad outage - this is the foundation of cross-region **disaster recovery** and GRS storage replication.

Step 4 - Check which VM SKUs support availability zones

```
az vm list-skus --location southeastasia --zone --size Standard_B --output table
```

The `--zone` flag filters to SKUs that can be deployed into availability zones; `--size Standard_B` narrows to the burstable B-series used in this course. In the `Zones` column you should see `1,2,3` for sizes like `Standard_B1s` and `Standard_B2s` - Southeast Asia has three availability zones.

```
az vm list-skus --location southeastasia --zone --resource-type virtualMachines --output table | head -20
```

Shows the broader list - note that virtually all mainstream VM sizes in this region are zone-capable.

Step 5 - See zones in the portal VM creation flow

47. Open `https://portal.azure.com`, search `virtual machines`, click `+ Create` → `Azure virtual machine`.
48. On the `Basics` tab set `Region` to `Southeast Asia`, then look at `Availability options` - select `Availability zone` and note you can pick `Zone 1, 2 or 3` (or multiple zones).
49. Change `Region` to a region without zones (e.g. `West Central US`) and observe the `availability-zone` option disappears.
50. Click `X` to close the wizard `without creating anything`.

Each availability zone is one or more physically separate datacenters with independent power, cooling and networking - deploying VMs across zones protects against a datacenter-level failure with an SLA of 99.99%.

Step 6 - Test your latency to Azure regions

51. Open `https://www.azure.speed.com` in a new tab.
52. On the `Latency Test` page, tick a handful of regions: `Southeast Asia`, `East Asia`, `Japan East`, `Australia East`, `West Europe`, `East US`.
53. Let the test run ~30 seconds and note the average latency to each.
54. Record: which region is fastest from Singapore? Roughly how much slower is Europe or the US?

Expect single-digit milliseconds to Southeast Asia and 150-250 ms to Europe/US - this is why you place resources in the region closest to your users, and why AZ-900 emphasises region selection for performance.

Step 7 - Discuss: zones vs region pairs - HA vs DR

Compare the two constructs and note the difference in your lab notes:

| | Availability zones | Region pairs |

|---|---|---|

| Distance | Separate datacenters, same region (km apart) | Different regions, 300+ miles apart |

| Protects against | Datacenter failure (power, cooling, network) | Regional disaster (earthquake, major outage) |

| Latency between | < 2 ms (synchronous replication possible) | Tens of ms (asynchronous replication) |

| Primary use | **High availability** (99.99% VM SLA) | **Disaster recovery** (GRS storage, cross-region failover) |

| Example | VMs in zones 1, 2, 3 behind a load balancer | Southeast Asia paired with East Asia |

Key takeaway: zones keep an application **up** through a local failure; pairs let a business **recover** from a regional catastrophe. Well-architected solutions use both.

Clean up

Nothing was created in this lab - the VM wizard was cancelled before deployment, and CLI commands were read-only. `rg-az900-labs` remains untouched and is reused in Lab 4.

What you learned

- How to list Azure regions and their paired regions with `az account list-locations` and JMESPath queries.
- The hierarchy geography → region → availability zone, and that Southeast Asia (Singapore) pairs with East Asia (Hong Kong).
- How to check zone support for VM SKUs with `az vm list-skus --zone`.
- That network latency grows with distance - measured live with azurespeed.com - driving region selection.
- Availability zones deliver high availability within a region (99.99% SLA), while region pairs enable disaster recovery across regions.

Lab 4 - Create a Windows Virtual Machine in the Portal

In this lab you deploy a Windows Server 2022 virtual machine through the portal, connect to it over RDP, install the IIS web server, expose it on port 80 through a network security group rule, and then stop (deallocate) it to see how IaaS compute billing works. This maps to the AZ-900 skill areas **Describe cloud concepts** (IaaS, shared responsibility) and **Describe Azure architecture and services** (virtual machines, networking basics). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 1 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-04-windows-vm-portal.md](#)

Step 1 - Start the Create VM wizard

55. In the portal search bar, type `virtual machines` and open **Virtual machines**.

56. Click **+ Create** → **Azure virtual machine**.

57. On the **Basics** tab set:

- Subscription: your subscription
- Resource group: `rg-az900-labs` (click **Create new** and enter it if it doesn't exist)
- Virtual machine name: `vm-az900-win01`
- Region: `Southeast Asia`
- Availability options: `No infrastructure redundancy required`
- Security type: `Trusted launch virtual machines` (default)
- Image: `Windows Server 2022 Datacenter: Azure Edition - x64 Gen2`
- Size: click **See all sizes** and choose `Standard\_B2s` (2 vCPUs, 4 GiB RAM - burstable, cheap)

The B-series is designed for workloads that idle most of the time - ideal for labs and free credit.

Step 2 - Set credentials and inbound ports

Still on the **Basics** tab:

58. Under **Administrator account**:

- Username: `azureadmin`
- Password: choose a strong password (12+ chars) and **write it down** - Azure never shows it again.

59. Under **Inbound port rules**:

- Public inbound ports: `Allow selected ports`
- Select inbound ports: `RDP (3389)`

60. Under **Licensing**, leave the Azure Hybrid Benefit checkbox unticked.

Opening RDP to the internet is acceptable for a short lab, but in production you would use Azure Bastion or restrict the source IP - note the portal's warning about this.

Step 3 - Review the remaining tabs, then create

61. Click **Next: Disks** - note the default **OS disk type**; change it to `Standard SSD (locally-redundant storage)` to keep costs low.

62. Click **Next: Networking** - observe that the wizard auto-creates a virtual network (`rg-az900-labs-vnet`), subnet, public IP (`vm-az900-win01-ip`) and NIC network security group with your RDP rule. Set **Delete public IP and NIC when VM is deleted** to checked.
63. Click through **Management**, **Monitoring**, **Advanced** - leave defaults.
64. Click **Review + create**. Check the **price per hour** shown for `Standard\_B2s` and confirm **Validation passed**.
65. Click **Create**.

A single "VM" is really a bundle of resources: compute, disk, NIC, public IP, NSG - you'll see them all land in **rg-az900-labs**.

Step 4 - Watch the deployment

66. The **Deployment** page opens automatically; watch resources turn green as ARM creates them (~2-3 minutes).
67. Click the **notifications bell** - the same deployment progress appears there.
68. When you see **Your deployment is complete**, click **Go to resource** to open the VM's **Overview** blade.
69. Note the **Public IP address** on the Overview - copy it.

Step 5 - Connect via RDP

70. On the VM's Overview blade, click **Connect** → select **Connect** (or **RDP** → **Download RDP file**).
71. Connect from your laptop:
 - **Windows**: open the downloaded `.rdp` file, or run `mstsc /v:<public-ip>`.
 - **macOS**: install **Windows App** (formerly Microsoft Remote Desktop) from the App Store, add a PC with the public IP, and connect.
72. Sign in as `azureadmin` with your password. Accept the certificate warning (it's a self-signed cert for the lab VM).
73. Windows Server desktop loads - Server Manager opens automatically.

You are now inside an IaaS VM: Microsoft manages the physical host; you manage the OS, patches and everything above - the shared responsibility model in action.

Step 6 - Install IIS with PowerShell inside the VM

74. Inside the RDP session, right-click **Start** → **Windows PowerShell (Admin)** (or **Terminal (Admin)**).
75. Run:

```
Install-WindowsFeature -Name Web-Server -IncludeManagementTools
```

Installs the IIS web server role; wait for **Success : True** in the output (1-2 minutes, no restart needed).

76. Verify locally: open **Microsoft Edge** inside the VM and browse to `http://localhost` - the blue IIS welcome page appears.
77. Leave the RDP session connected (or minimise it).

Step 7 - Open port 80 in the network security group

The IIS page works inside the VM but is not reachable from the internet yet - port 80 is blocked by the NSG.

78. Back in the portal, on the VM's left menu open **Networking** → **Network settings**.

79. Under **Rules**, click **+ Create port rule** → **Inbound port rule** and set:

- Source: `Any`
- Source port ranges: ``
- Destination: `Any`
- Service: `HTTP` (sets Destination port `80`, Protocol `TCP`)
- Action: `Allow`
- Priority: `1010`
- Name: `Allow-HTTP-80`

80. Click **Add** and wait for the "Created security rule" notification.

NSGs are stateful packet filters evaluated by priority (lower number = evaluated first) - a core Azure networking concept.

Step 8 - Browse the site from the internet

81. On your **own laptop's** browser (not inside the VM), go to `http://<public-ip>` using the IP copied in Step 4.

82. The default **IIS Windows Server** welcome page loads.

You have deployed a working public web server on IaaS: VM + OS + web role + firewall rule - everything a physical server needs, provisioned in minutes.

Step 9 - Stop (deallocate) the VM and observe billing behaviour

83. Close your RDP session (sign out inside the VM, or just close the window).

84. On the VM's **Overview** blade, click **Stop** and confirm. (Optionally leave "reserve public IP" unticked.)

85. Watch the **Status** field change: `Stopping` → **Stopped (deallocated)**.

86. Key observation: **Stopped (deallocated)** means the compute hardware is released back to Azure and **you stop paying for the VM's compute time**. You still pay small charges for the OS disk (storage) and any static public IP. Shutting down from inside Windows only reaches `Stopped` - still billed; always stop from the portal/CLI to deallocate.

Clean up

Keep the VM for now if you are continuing the course the same day - later labs reuse **rg-az900-labs**, and a **deallocated** VM costs only its disk (~a few cents/day).

When you no longer need it:

87. Open **Virtual machines** → `vm-az900-win01` → **Delete**.

88. In the delete pane, tick the checkboxes to also delete the **OS disk**, **Network interface** and **Public IP address**, then confirm.

89. **Do not delete the resource group `rg-az900-labs`** - it is reused by every remaining lab (deleted only in Lab 16).

What you learned

- IaaS in practice: a VM bundles compute, managed disk, NIC, public IP and NSG, all deployed together by ARM into a resource group.
- How to size a VM for cost (`Standard\_B2s` burstable) and connect over RDP from Windows (mstsc) or macOS (Windows App).
- Shared responsibility: Azure runs the host; you install and manage the OS workload (IIS via `Install-WindowsFeature`).
- NSG inbound rules control traffic by priority - the site was unreachable until port 80 was allowed.
- `Stopped (deallocated)` releases compute and stops compute billing (disks and static IPs still bill) - a key cost-management behaviour on the AZ-900 exam.

Lab 5 - Create a Linux VM with the Azure CLI

In this lab you provision an Ubuntu virtual machine entirely from the command line, open a port, SSH in, install the NGINX web server, check its power state, and cleanly delete it - experiencing the CLI as a full alternative to the portal. This maps to the AZ-900 skill areas **Describe Azure architecture and services** (virtual machines, IaaS) and **Describe Azure management and governance** (Azure CLI as a management tool). Run in Azure Cloud Shell - <https://shell.azure.com>

Note: Day 1 | Domain 2 - Describe Azure architecture and services | Source:
[labs/lab-05-linux-vm-cli.md](#)

Step 1 - Ensure the resource group exists

```
az group create --name rg-az900-labs --location southeastasia --tags course=az900
```

Idempotent - succeeds whether or not the group already exists from earlier labs, so this lab is self-contained.

Step 2 - Create the Linux VM

```
az vm create \
  --resource-group rg-az900-labs \
  --name vm-az900-linux01 \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --admin-username azureuser \
  --generate-ssh-keys
```

One command creates the VM plus its VNet, subnet, public IP, NIC and NSG (defaults are generated when not specified). **--generate-ssh-keys** creates a key pair in `~/.ssh` (or reuses an existing one) and installs the public key - no password needed. After 1-2 minutes it returns JSON; note the **publicIpAddress** field. **Standard_B1s** (1 vCPU, 1 GiB) is the cheapest general-purpose size - perfect for a lab web server.

Step 3 - Open port 80 for web traffic

```
az vm open-port --resource-group rg-az900-labs --name vm-az900-linux01 --port 80
```

Adds an allow rule for TCP 80 to the VM's network security group - the CLI equivalent of the portal NSG rule you created in Lab 4. SSH (22) was already opened by **az vm create** for Linux images.

Step 4 - Get the VM's IP addresses

```
az vm list-ip-addresses --resource-group rg-az900-labs --name vm-az900-linux01 -o table
```

Shows the VM's public IP and private IP in a table. Copy the **PublicIPAddresses** value - you will use it for SSH and the browser test. Store it in a variable to avoid retyping:

```
IP=$(az vm list-ip-addresses -g rg-az900-labs -n vm-az900-linux01 \
  --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv)
echo $IP
```

Step 5 - SSH into the VM from Cloud Shell

```
ssh azureuser@$IP
```

Type **yes** at the host-key prompt. You land on the Ubuntu shell (`azureuser@vm-az900-linux01:~$`) - Cloud Shell already holds the private key generated in Step 2, so login is passwordless. This is the Linux equivalent of the RDP connection you made in Lab 4.

Step 6 - Install NGINX inside the VM

```
sudo apt update && sudo apt install -y nginx
```

Refreshes the package index and installs the NGINX web server; the service starts automatically on install. Verify from inside the VM:

```
curl http://localhost
```

Returns the **Welcome to nginx!** HTML - the web server is running locally.

Step 7 - Exit and test from outside

```
exit
```

Returns you to Cloud Shell. Now test the public endpoint:

```
curl http://$IP
```

Returns the same **Welcome to nginx!** page - traffic now flows internet → public IP → NSG (port 80 rule from Step 3) → VM. You can also open **http://<public-ip>** in your laptop's browser.

Step 8 - Check the VM's power state

```
az vm show --resource-group rg-az900-labs --name vm-az900-linux01 --show-details --query powerState -o tsv
```

--show-details adds runtime info (power state, IPs) to the model view; expect **VM running**. Try deallocating and re-checking to see billing-relevant states:

```
az vm deallocate -g rg-az900-labs -n vm-az900-linux01
az vm show -g rg-az900-labs -n vm-az900-linux01 --show-details --query powerState -o tsv
```

Now shows **VM deallocated** - compute billing has stopped, exactly as you observed in the portal in Lab 4.

Step 9 - Delete the VM and its attached resources

```
az vm delete --resource-group rg-az900-labs --name vm-az900-linux01 --yes
```

Deletes the VM **compute resource only** - by default the NIC, public IP, OS disk and NSG it used are left behind as **orphaned resources** that can still incur cost (disk, static IP). List what remains:

```
az resource list --resource-group rg-az900-labs -o table
```

Delete the leftovers by name (adjust names to match the table output):

```
az network nic delete -g rg-az900-labs -n vm-az900-linux01VMNic
az network public-ip delete -g rg-az900-labs -n vm-az900-linux01PublicIP
az network nsg delete -g rg-az900-labs -n vm-az900-linux01NSG
az disk list -g rg-az900-labs --query "[?contains(name,'linux01')].name" -o tsv | \
  xargs -I{} az disk delete -g rg-az900-labs -n {} --yes
```

Alternative: delete the VM in the **portal** instead, where the delete pane offers "**Delete with VM**" checkboxes for the disk, NIC and public IP in one action. Orphaned resources are a classic real-world (and exam) cost-management gotcha.

Clean up

Steps 8-9 already deallocated and deleted the VM and its attached resources. Verify nothing billable is left:

```
az resource list --resource-group rg-az900-labs -o table
```

Only resources from other labs (e.g. Lab 4's VM and VNet, if you kept them) should remain. **Keep the resource group `rg-az900-labs`** - it is reused by the remaining labs and deleted only in Lab 16.

What you learned

- One `az vm create` command provisions a complete IaaS stack - VM, VNet, NIC, public IP, NSG - with sensible defaults and SSH keys via `--generate-ssh-keys`.
- `az vm open-port` is the CLI shortcut for adding an NSG inbound rule.
- Managing a Linux VM end to end: SSH from Cloud Shell, `apt install nginx`, and verifying with `curl` inside and outside the VM.
- `az vm show --show-details --query powerState` distinguishes `VM running` from `VM deallocated` (no compute charges).
- `az vm delete` leaves orphaned NICs, IPs and disks that still cost money - clean them up explicitly, or use the portal's "Delete with VM" checkboxes.

Lab 6 - Create a Web App with Azure App Service

In this lab you create a fully managed web app on Azure App Service, deploy a simple HTML page to it with a single CLI command, and explore the scaling options that PaaS gives you without touching a single server. This maps to the AZ-900 skill area **Describe Azure architecture and services** - Platform as a Service (PaaS) compute. Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 1 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-06-app-service-webapp.md](#)

Step 1 - Create the resource group (if it doesn't exist)

90. In the portal search bar, type **Resource groups** and open it.

91. Click **+ Create** and fill in:

- Subscription: your free subscription
- Resource group: ``rg-az900-labs``
- Region: ``Southeast Asia``

92. Click **Review + create** → **Create**. (Skip this step if ``rg-az900-labs`` already exists from an earlier lab.)

All labs in this course share this resource group so cleanup stays simple.

Step 2 - Create the Web App

93. Click **Create a resource** (top-left menu) → search **Web App** → **Create** → **Web App**.

94. On the **Basics** tab enter:

| Field | Value |

|---|---|

| Subscription | your free subscription |

| Resource group | `rg-az900-labs` |

| Name | `app-az900-<yourinitials><random>` (e.g. `app-az900-tan4821` - must be globally unique) |

| Publish | `Code` |

| Runtime stack | `Node 20 LTS` (or `PHP 8.2`) |

| Operating System | `Linux` |

| Region | `Southeast Asia` |

95. Under **Pricing plans**, click **Create new** for the Linux Plan, name it ``plan-az900-free``, then click **Explore pricing plans** and select **Free F1** → **Select**.

96. Click **Review + create** → **Create** and wait for deployment (about 1 minute).

The name becomes part of a public URL (`<name>.azurewebsites.net`), which is why it must be globally unique.

Step 3 - Browse the default placeholder page

97. When deployment completes, click **Go to resource**.
98. On the **Overview** blade, find **Default domain** and click the `https://app-az900-....azurewebsites.net` link.
99. A default "Your web app is running and waiting for your content" page appears.

Notice you never created a VM, installed a web server, or opened a firewall port - App Service manages all of that for you.

Step 4 - Open Cloud Shell and create a simple page

100. Click the **Cloud Shell** icon (>) in the top toolbar of the portal and choose **Bash** (create storage if prompted, accept defaults).
101. Create a folder and an `index.html`:

```
mkdir site && cd site
code index.html
```

The `code` command opens the built-in Cloud Shell editor. Paste this content:

```
<h1>Hello from Azure App Service!</h1>
<p>Deployed by <your name> - AZ-900 Lab 6</p>
```

102. Save with **Ctrl+S** and close the editor with **Ctrl+Q**.

Step 5 - Zip and deploy the page

```
zip site.zip index.html
```

This packages your file - App Service accepts zip packages as a deployment source.

```
az webapp deploy --resource-group rg-az900-labs --name app-az900-  
site.zip --type zip
```

`--type zip` tells the deployment engine to extract the archive into the web root (`/home/site/wwwroot`). Wait for the JSON output showing `"provisioningState": "Succeeded"`.

Step 6 - Verify the deployment

103. Go back to the browser tab with your web app and refresh (Ctrl+F5 to bypass cache).
104. Your "Hello from Azure App Service!" page replaces the placeholder.

One command took your code from Cloud Shell to a live, HTTPS-enabled public website - this is the core PaaS value proposition.

Step 7 - Explore Scale up (App Service plan)

105. On your web app's left menu, under **Settings**, click **Scale up (App Service plan)**.
106. Browse the tiers: **Dev/Test** (F1, B1), **Production** (P0v3, P1v3...). Note what higher tiers add: custom domains + TLS, staging slots, autoscale, more CPU/RAM.
107. **Do NOT change the tier** - stay on F1. Close the blade without applying.

Scaling **up** means moving to a bigger instance size (vertical scaling).

Step 8 - Explore Scale out (App Service plan)

108. Under **Settings**, click **Scale out (App Service plan)**.

109. Observe that on the Free F1 tier the instance count is fixed at 1, and rules-based/automatic scaling requires a paid tier.
110. ****Do NOT scale**** - just note that on Standard and above you could set autoscale rules (e.g. add an instance when CPU > 70%).

Scaling **out** means adding more instances of the same size (horizontal scaling) - the preferred cloud pattern for handling variable load.

Clean up

Keep the resource group **rg-az900-labs** - it is reused until Lab 16. Delete only this lab's resources:

```
az webapp delete --resource-group rg-az900-labs --name app-az900-<yourinitials><random>
az appservice plan delete --resource-group rg-az900-labs --name plan-az900-free --yes
```

Or in the portal: open **rg-az900-labs**, tick the web app and the App Service plan, click **Delete** and confirm. (F1 is free, but deleting keeps the resource group tidy.)

What you learned

- App Service is ****PaaS****: Azure manages the OS, web server, patching, and TLS - you manage only your code (shared responsibility model).
- An ****App Service plan**** defines the compute (tier, size, region) that one or more web apps run on.
- Zip deploy via ``az webapp deploy`` is a simple, repeatable way to publish code to App Service.
- ****Scale up**** = bigger instances (vertical); ****scale out**** = more instances (horizontal); higher tiers unlock autoscale.
- App names must be globally unique because they form part of the public ``*.azurewebsites.net`` DNS name.

Lab 7 - Deploy a Container with Azure Container Instances

In this lab you run a public Docker container image on Azure Container Instances (ACI) with a single command, browse the running app on a public URL, and inspect its logs - all without provisioning any VM. This maps to the AZ-900 skill area **Describe Azure architecture and services** - container services and serverless-style compute. Run in Azure Cloud Shell - <https://shell.azure.com>

Note: Day 1 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-07-container-instances.md](#)

Step 1 - Open Cloud Shell and ensure the resource group exists

Open <https://shell.azure.com>, choose **Bash**, then run:

```
az group create --name rg-az900-labs --location southeastasia
```

This is idempotent - if **rg-az900-labs** already exists from an earlier lab, the command simply returns it.

Step 2 - Create the container instance

```
az container create \  
  --resource-group rg-az900-labs \  
  --name aci-az900-hello \  
  --image mcr.microsoft.com/azuredocs/aci-helloworld \  
  --dns-name-label az900-  
<yourinitials><random> \  
  --ports 80 \  
  --os-type Linux \  
  --cpu 1 \  
  --memory 1
```

--image pulls Microsoft's sample hello-world web app from the Microsoft Container Registry; **--dns-name-label** must be unique within the region because it becomes part of a public FQDN; **--cpu 1 --memory 1** requests 1 vCPU and 1 GB RAM. The command takes about a minute and returns JSON with **"provisioningState": "Succeeded"**.

Note how fast this is compared to a VM - no OS to deploy, just a container image to pull and start.

Step 3 - Get the public FQDN

```
az container show \  
  --resource-group rg-az900-labs \  
  --name aci-az900-hello \  
  --query ipAddress.fqdn \  
  --output tsv
```

--query uses JMESPath to extract just the fully qualified domain name, e.g. **az900-tan4821.southeastasia.azurecontainer.io**.

Step 4 - Browse the running container

Open a browser tab and go to **<http://<the-fqdn-from-step-3>>**.

You should see the blue **"Welcome to Azure Container Instances!"** page. The container is serving real HTTP traffic on a public IP and DNS name that Azure provisioned for you.

Step 5 - View the container logs

```
az container logs \  
  --resource-group rg-az900-labs \  
  --name aci-az900-hello
```

You see the Node.js app's stdout, including a log line for the HTTP GET request your browser just made. Container logs are the primary troubleshooting tool for ACI - there is no server to SSH into.

Step 6 - Restart the container group

```
az container restart \  
  --resource-group rg-az900-labs \  
  --name aci-az900-hello
```

Refresh the browser tab - the page is back within seconds. Compare this to a VM reboot, which typically takes minutes: containers restart in seconds because there is no operating system to boot, only a process.

Step 7 - Compare containers vs VMs vs PaaS

No commands here - check your understanding against this table:

Virtual machine (Lab 4/5) Container (this lab) App Service PaaS (Lab 6)			
--- --- --- ---			
You manage OS, patching, runtime, app		Container image, app App code only	
Startup time Minutes		Seconds Seconds (already-running platform)	
Density/efficiency 1 OS per VM		Many containers share one OS kernel Multi-tenant platform	
Isolation Strong (hardware-virtualised)		Process-level Platform-managed	
Best for Lift-and-shift, full OS control		Portable microservices, burst jobs Standard web apps and APIs	

Containers are lightweight virtualisation: they virtualise the OS rather than the hardware, which is why they start faster and pack more densely than VMs.

Step 8 - Delete the container group

```
az container delete \  
  --resource-group rg-az900-labs \  
  --name aci-az900-hello \  
  --yes
```

ACI bills per second while the container group exists, so deleting it immediately stops all charges.

Clean up

Step 8 already removed the only resource this lab created. Verify nothing is left:

```
az container list --resource-group rg-az900-labs --output table
```

The output should be empty. Keep the resource group **rg-az900-labs** itself - it is reused until Lab 16.

What you learned

- **Azure Container Instances** runs containers on demand with no VMs or orchestrator to manage - the fastest way to run a container in Azure.
- Containers start in **seconds** because they share the host OS kernel instead of booting their own, giving higher density than VMs.
- A `--dns-name-label` gives a container group a public FQDN of the form `<label>.<region>.azurecontainer.io`.
- `az container logs` replaces SSH for troubleshooting - you observe the app, not a server.
- Choosing between **IaaS (VMs)**, containers, and **PaaS** is about how much of the stack you want to manage - a core AZ-900 exam theme.

Lab 8 - Create a Virtual Network and Test Private Connectivity

In this lab you build a virtual network with two subnets, place a "web" VM with a public IP in one and a "db" VM with no public IP in the other, then prove the two can talk privately over the VNet. This maps to the AZ-900 skill area **Describe Azure architecture and services** - virtual networking, public and private endpoints. Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 1 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-08-virtual-network.md](#)

Step 1 - Ensure the resource group exists

Open **Cloud Shell** (>_ icon, **Bash**) in the portal and run:

```
az group create --name rg-az900-labs --location southeastasia
```

Idempotent - returns the existing group if you already created it in an earlier lab.

Step 2 - Create the virtual network and subnets

```
az network vnet create \
  --resource-group rg-az900-labs \
  --name vnet-az900 \
  --address-prefix 10.0.0.0/16 \
  --subnet-name subnet-web \
  --subnet-prefix 10.0.1.0/24
```

This creates the VNet with a /16 address space (65,536 private addresses) and the first subnet. Now add the second subnet:

```
az network vnet subnet create \
  --resource-group rg-az900-labs \
  --vnet-name vnet-az900 \
  --name subnet-db \
  --address-prefix 10.0.2.0/24
```

Subnets segment the address space so you can apply different security rules to different tiers (web vs database).

Step 3 - Inspect the VNet in the portal

111. In the portal search bar, type **Virtual networks** and open **vnet-az900**.
112. On the left menu click **Settings → Subnets** and confirm:

| Subnet | Address range |

|---|---|

| subnet-web | 10.0.1.0/24 |

| subnet-db | 10.0.2.0/24 |

Note Azure reserves 5 IPs in every subnet (network, broadcast, gateway, DNS), so a /24 gives 251 usable addresses.

Step 4 - Create the web VM (with public IP)

Back in Cloud Shell:

```
az vm create \  
  --resource-group rg-az900-labs \  
  --name vm-az900-web \  
  --image Ubuntu2204 \  
  --size Standard_B1s \  
  --vnet-name vnet-az900 \  
  --subnet subnet-web \  
  --admin-username azureuser \  
  --generate-ssh-keys \  
  --public-ip-sku Standard
```

--generate-ssh-keys creates (or reuses) an SSH key pair in Cloud Shell so no password is needed. Note the **publicIpAddress** in the output - you will SSH to it in Step 6.

Step 5 - Create the db VM (NO public IP)

```
az vm create \  
  --resource-group rg-az900-labs \  
  --name vm-az900-db \  
  --image Ubuntu2204 \  
  --size Standard_B1s \  
  --vnet-name vnet-az900 \  
  --subnet subnet-db \  
  --admin-username azureuser \  
  --generate-ssh-keys \  
  --public-ip-address ""
```

The empty **--public-ip-address ""** means this VM gets **no public IP** - it is reachable only from inside the VNet. Note its **privateIpAddress** in the output (it will be **10.0.2.4** or similar).

A database server with no public endpoint is a real-world security best practice: it removes the entire internet as an attack surface.

Step 6 - SSH to the web VM

```
ssh azureuser@<publicIpAddress-of-vm-az900-web>
```

Type **yes** to accept the host key. Your prompt changes to **azureuser@vm-az900-web** - you are now inside the VNet.

Step 7 - Test private connectivity to the db VM

From the **vm-az900-web** SSH session:

```
ping -c 4 10.0.2.4
```

Replace **10.0.2.4** with the private IP you noted in Step 5. You should see 4 replies with ~1 ms latency - the two subnets route to each other automatically over the VNet.

Now hop to the db VM over its private IP (SSH agent forwarding is not set up, so use password-less hop only if keys were shared; otherwise the connection attempt itself proves TCP/22 reachability):

```
ssh -o ConnectTimeout=5 azureuser@10.0.2.4
```

Even a "Permission denied (publickey)" response proves the private network path works - the SSH service answered. Type `exit` to leave the web VM.

You just proved: the db VM is invisible from the internet but fully reachable from inside the virtual network.

Step 8 - View the NSG default rules

113. In the portal, search **Network security groups** and open **vm-az900-webNSG** (created automatically with the VM).

114. Click **Settings → Inbound security rules** and observe the default rules at priority 65000+:

- ``AllowVnetInBound`` - traffic within the VNet is allowed (this is why the ping worked).
- ``AllowAzureLoadBalancerInBound`` - health probes allowed.
- ``DenyAllInBound`` - everything else from the internet is denied.

115. Note the higher-priority rule allowing SSH (port 22) that ``az vm create`` added for the web VM.

NSGs are stateful packet filters evaluated by priority (lower number wins) - a core exam concept.

Step 9 - Public vs private endpoints (discussion)

No commands - relate what you built to exam terminology:

- ``vm-az900-web``'s public IP is a **public endpoint**: reachable from the internet, protected only by NSG rules.
- ``vm-az900-db``'s private IP is effectively a **private endpoint** pattern: services (like Azure SQL or Storage) can likewise be given a private IP inside your VNet via **Azure Private Link**, so traffic never crosses the public internet.

Minimising public endpoints is a pillar of the defense-in-depth model tested in AZ-900.

Clean up

Delete both VMs, their NICs/disks/IPs, and the VNet (keep resource group `rg-az900-labs` - it is reused until Lab 16):

```
az vm delete --resource-group rg-az900-labs --name vm-az900-web --yes
az vm delete --resource-group rg-az900-labs --name vm-az900-db --yes
az network nic delete --resource-group rg-az900-labs --name vm-az900-webVMNic
az network nic delete --resource-group rg-az900-labs --name vm-az900-dbVMNic
az network nsg delete --resource-group rg-az900-labs --name vm-az900-webNSG
az network nsg delete --resource-group rg-az900-labs --name vm-az900-dbNSG
az network public-ip delete --resource-group rg-az900-labs --name vm-az900-webPublicIP
az network vnet delete --resource-group rg-az900-labs --name vnet-az900
```

Then delete the leftover OS disks:

```
az disk list --resource-group rg-az900-labs --query "[].name" -o tsv
az disk delete --resource-group rg-az900-labs --name <each-disk-name> --yes
```

VMs bill per second while allocated, so do this before ending the session.

What you learned

- An **Azure Virtual Network** is your private IP space in the cloud; **subnets** segment it by tier or workload.
- VMs in different subnets of the same VNet communicate automatically over **private IPs** - no extra routing needed.
- A VM (or service) **without a public IP** is unreachable from the internet - a fundamental security control.
- **NSG default rules** allow intra-VNet traffic and deny all other inbound traffic; rules are evaluated by priority.
- **Public endpoints** expose services to the internet; **private endpoints/Private Link** keep service traffic inside your VNet.

Lab 9 - Create a Storage Account and Work with Blob Storage

In this lab you create a general-purpose storage account, upload a blob, switch its access tier, and share it securely with a time-limited SAS URL - then list the same blobs from the CLI. This maps to the AZ-900 skill area **Describe Azure architecture and services** - storage services, tiers, and redundancy. Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-09-storage-blobs.md](#)

Step 1 - Create the storage account

116. Click **Create a resource** → search **Storage account** → **Create**.

117. On the **Basics** tab enter:

| Field | Value |

|---|---|

| Subscription | your free subscription |

| Resource group | **rg-az900-labs** (click **Create new** if it doesn't exist) |

| Storage account name | **staz900<yourinitials><random>** (e.g. **staz900tan4821** - lowercase letters/numbers only, globally unique) |

| Region | **Southeast Asia** |

| Performance | **Standard** |

| Redundancy | **Locally-redundant storage (LRS)** |

118. Leave all other tabs at defaults, click **Review + create** → **Create**, then **Go to resource**.

LRS keeps 3 copies of your data within one datacenter - the cheapest redundancy option, ideal for labs.

Step 2 - Create a private blob container

119. On the storage account's left menu, click **Data storage** → **Containers**.

120. Click **+ Container** and enter:

- Name: ``labfiles``
- Anonymous access level: ``Private (no anonymous access)``

121. Click **Create**.

Private access means every request must be authorised - by Entra ID, an account key, or a SAS token.

Step 3 - Upload a file via the portal

122. On your computer, create a small text file ``hello.txt`` containing one line, e.g. ``Hello from Azure Blob Storage - AZ-900 Lab 9`` (any small text or image file works).

123. Open the `labfiles` container → click **Upload**.
124. Browse to your file, expand **Advanced** and note the defaults (Block blob, access tier **Hot**), then click **Upload**.

Block blobs are the standard blob type for files; page blobs (VM disks) and append blobs (logs) are the other two.

Step 4 - Change the blob's access tier Hot → Cool

125. In the `labfiles` container, click the blob `hello.txt` to open its properties.
126. Click **Change tier** (top toolbar), set **Access tier** = `Cool`, click **Save**.
127. Re-open the blob's properties and confirm **Access tier: Cool** and note the tier-change timestamp.

Cool storage costs less per GB but more per access, with a 30-day minimum - tiers let you match cost to how often data is read. (Cold and Archive tiers go further; Archive requires rehydration before reading.)

Step 5 - Try to open the blob URL anonymously

128. On the blob's **Overview**, copy the **URL** (e.g. `https://staz900tan4821.blob.core.windows.net/labfiles/hello.txt`).
129. Paste it into a **private/incognito** browser window.
130. You get an XML error: `ResourceNotFound` / `The specified resource does not exist` - anonymous access is blocked because the container is private.

This is the expected, secure default: a URL alone is not enough to read private data.

Step 6 - Generate a SAS URL and open it

131. Back on the blob's page, click the **Generate SAS** tab.
132. Configure:
 - Signing method: `Account key`
 - Permissions: `Read` only
 - Start: now; Expiry: 1 hour from now
133. Click **Generate SAS token and URL** and copy the **Blob SAS URL**.
134. Paste the SAS URL into the same private window - the file now opens/downloads.

A **shared access signature** grants delegated, scoped, time-limited access without sharing your account keys. When the expiry passes, the same URL stops working.

Step 7 - List blobs from the CLI

Open **Cloud Shell** (Bash) and run:

```
az storage blob list \
  --account-name staz900<yourinitials><random> \
  --container-name labfiles \
  --auth-mode login \
  --output table
```

`--auth-mode login` uses your Entra ID sign-in. If you get an authorization error, your account needs the **Storage Blob Data Reader** role on the storage account (Owner of the subscription is not automatically a data-plane reader). Alternative - authenticate with the account key instead:

```
az storage account keys list --resource-group rg-az900-labs --account-name
staz900<yourinitials><random> --query "[0].value" -o tsv
az storage blob list \
  --account-name staz900<yourinitials><random> \
  --container-name labfiles \
  --auth-mode key \
  --output table
```

This illustrates the difference between **management plane** (ARM/RBAC roles like Owner) and **data plane** (data access roles or keys) - a subtle point AZ-900 touches on under RBAC.

Step 8 - Explore the Storage browser blade

135. On the storage account's left menu, click **Storage browser**.
136. Expand **Blob containers** → **labfiles** and view your blob; note you can also browse **File shares**, **Queues**, and **Tables** from here.

One storage account hosts four services: Blobs (objects), Files (SMB shares), Queues (messaging), Tables (NoSQL key-value).

Step 9 - Review redundancy options

137. On the storage account's left menu, click **Data management** → **Redundancy**.
138. Open the redundancy dropdown and compare (don't change it):

| Option | Copies | Protects against |

|---|---|---|

| LRS | 3 in one datacenter | Drive/rack failure |

| ZRS | 3 across availability zones | Datacenter failure |

| GRS | 6 (3 local + 3 in paired region) | Regional disaster |

| GZRS | 6 (zones + paired region) | Zone + regional disaster |

139. Note the map showing the paired region for GRS options.

Choosing redundancy is a cost-vs-durability trade-off - a classic AZ-900 exam question.

Clean up

The storage account costs almost nothing while nearly empty, so you may keep it - it will be removed with `rg-az900-labs` in Lab 16. To delete it now instead:

```
az storage account delete --resource-group rg-az900-labs --name staz900<yourinitials><random> --yes
```

Keep the resource group `rg-az900-labs` itself - it is reused until Lab 16.

What you learned

- A **storage account** is a globally-unique namespace hosting Blobs, Files, Queues, and Tables.

- ****Access tiers**** (Hot/Cool/Cold/Archive) trade storage cost against access cost for infrequently used data.
- Private containers block anonymous access; a ****SAS token**** grants scoped, time-limited access without exposing account keys.
- Data-plane access uses either ****Entra ID roles**** (e.g. Storage Blob Data Reader) or ****account keys**** - separate from management-plane RBAC.
- ****Redundancy options**** (LRS → ZRS → GRS → GZRS) increase durability and cost by copying data across zones and paired regions.

Lab 10 - Create an Azure SQL Database

In this lab you deploy a fully managed Azure SQL Database preloaded with the AdventureWorksLT sample data and query it directly from the browser - no database server to install, patch, or back up. This maps to the AZ-900 skill area **Describe Azure architecture and services** - database services and the PaaS shared-responsibility model. Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-10-azure-sql.md](#)

Step 1 - Start the SQL Database wizard

140. Click **Create a resource** → search **SQL Database** → **Create** → **SQL Database**.

141. On the **Basics** tab set:

- Subscription: your free subscription
- Resource group: `rg-az900-labs` (create it if it doesn't exist)
- Database name: `db-az900`

Step 2 - Create a new logical server

142. Under **Server**, click **Create new** and enter:

| Field | Value |

|---|---|

| Server name | `sql-az900-<yourinitials><random>` (e.g. `sql-az900-tan4821` - globally unique) |

| Location | `Southeast Asia` |

| Authentication method | `Use SQL authentication` |

| Server admin login | `sqladmin` |

| Password | a strong password - write it down, you need it in Step 6 |

143. Click **OK**.

The logical server is a management boundary (logins, firewall rules) - it is not a VM you can log into.

Step 3 - Choose a cheap compute + storage tier

144. Back on **Basics**, set **Workload environment** = `Development` if offered.

145. Under **Compute + storage**, click **Configure database**.

146. Choose one of the two lab-friendly options:

- **Service tier** = `Basic` (5 DTUs, 2 GB) - simplest and about USD 5/month, or
- **Service tier** = `General Purpose (Serverless)` with **Auto-pause delay** = 1 hour and min/max vCores at the lowest values - bills per second of use and pauses when idle.

147. Click **Apply**. On **Basics**, set **Backup storage redundancy** = `Locally-redundant backup storage`.

Serverless with auto-pause is the cloud consumption model in miniature: pay only while the database is active.

Step 4 - Configure networking

148. Click **Next: Networking** and set:

- Connectivity method: `Public endpoint`
- Allow Azure services and resources to access this server: `Yes`
- Add current client IP address: `Yes`

149. Leave **Connection policy** at Default.

By default a SQL server's firewall blocks everything; these two rules admit Azure services (needed by Query editor) and your own machine.

Step 5 - Load sample data and create

150. Click **Next: Security** - leave defaults (make sure **Microsoft Defender for SQL** = `Not now` to avoid charges).

151. Click **Next: Additional settings** and set:

- Use existing data: `Sample` (this loads the **AdventureWorksLT** database; accept the collation prompt)

152. Click **Review + create** → **Create**. Deployment takes 2-5 minutes; then click **Go to resource**.

Step 6 - Query the database from the browser

153. On the **db-az900** database blade, click **Query editor (preview)** in the left menu.

154. Sign in with **SQL server authentication**: login `sqladmin` and your password from Step 2.

155. In the query pane, run:

```
SELECT TOP 10 Name, ListPrice FROM SalesLT.Product ORDER BY ListPrice DESC;
```

156. Click **Run** - the 10 most expensive AdventureWorks products appear in the Results grid.

You just queried a production-grade SQL database entirely from a browser. If you chose serverless and the DB auto-paused, the first query may take ~30 seconds to resume - that's the trade-off for paying nothing while paused.

Step 7 - Observe what PaaS removed from your plate

157. On the database's **Overview** blade note: compute tier, storage used, and the server name `sql-az900-....database.windows.net`.

158. Look at the left menu: there is no "Restart VM", no OS, no patching schedule. Under **Data management** → **Backups** (on the server) note that automated backups with point-in-time restore are already running - you configured nothing.

With SQL on an Azure VM (IaaS) you would patch Windows and SQL Server yourself; with Azure SQL Database (PaaS), Microsoft handles the OS, engine patching, HA, and backups.

Step 8 - Check the server firewall

159. From the database **Overview**, click the **Server name** link to open the logical server.

160. In the left menu click **Security → Networking**.
161. Under **Firewall rules**, see the rule with your client IP (added in Step 4) and the checkbox **Allow Azure services and resources to access this server**.
162. Note you could delete the client rule to instantly cut off your own access - the firewall is the outer gate before SQL authentication even happens.

Defense in depth: network firewall first, then authentication, then database permissions.

Clean up

Delete the database and the logical server (keep resource group **rg-az900-labs** - it is reused until Lab 16). In Cloud Shell:

```
az sql db delete --resource-group rg-az900-labs --server sql-az900-  
db-az900 --yes  
az sql server delete --resource-group rg-az900-labs --name sql-az900-  
--yes
```

Or in the portal: open **rg-az900-labs**, tick **db-az900** and the SQL server, click **Delete**. Basic tier bills daily, so don't leave it running.

What you learned

- **Azure SQL Database** is a PaaS relational database: Microsoft manages the OS, patching, high availability, and automated backups.
- A **logical SQL server** is an administrative boundary for logins and firewall rules, not a machine.
- **DTU (Basic)** vs **vCore serverless with auto-pause** are two purchasing models - serverless embodies pay-per-use consumption pricing.
- The **server firewall** blocks all access by default; you explicitly allow client IPs and Azure services.
- **Query editor** in the portal lets you run T-SQL with zero client tooling - useful for quick verification and demos.

Lab 11 - Explore Microsoft Entra ID: Users and Groups

In this lab you explore your tenant in Microsoft Entra ID (formerly Azure AD), create a user and a security group, and sign in as that user to see what "authenticated but not authorised" looks like. This maps to the AZ-900 skill area **Describe Azure architecture and services** - identity, access, and the difference between authentication and authorization. Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 2 - Describe Azure architecture and services | Source: [labs/lab-11-entra-id.md](#)

Step 1 - WARNING: use your own tenant, not your company's

Do this lab only in the tenant of your own Azure free account. If your portal session is signed into a company or school tenant, creating users and groups there may violate policy and confuse your IT department.

163. Click your account avatar (top-right) → **Switch directory**.
164. Confirm the current directory is your personal free-account tenant (usually named "Default Directory" with a domain like `

Step 2 - Open Microsoft Entra ID and note your tenant details

165. In the portal search bar, type **Microsoft Entra ID** and open it.
166. On the **Overview** blade, write down:
 - **Tenant name** (e.g. `Default Directory`)
 - **Primary domain** - ` - **Tenant ID** - a GUID
167. Note the counts of Users, Groups, and Applications.

Every Azure subscription trusts exactly one Entra tenant - the tenant is the identity boundary; the subscription is the billing/resource boundary.

Step 3 - Create a new user

168. In Entra ID, click **Manage** → **Users** → **+ New user** → **Create new user**.
169. On the **Basics** tab enter:

| Field | Value |

|---|---|

| User principal name | **az900.learner @ <yourtenant>.onmicrosoft.com** |

| Display name | **AZ900 Learner** |

| Password | tick **Auto-generate password**, then click the copy icon and **save the password** - you need it in Step 6 |

170. Click **Review + create** → **Create**.
171. Refresh the Users list and confirm `AZ900 Learner` appears with source "Cloud".

This is a cloud-only identity that exists purely in your tenant - no on-premises AD involved.

Step 4 - Create a security group

172. In Entra ID, click **Manage** → **Groups** → **+ New group**.
173. Enter:

- Group type: ``Security``
- Group name: ``grp-az900-learners``
- Membership type: ``Assigned``

174. Click **Create**.

Assigned membership means an admin adds members manually; **Dynamic** groups instead auto-populate from rules (e.g. department = Sales) - a Premium feature.

Step 5 - Add the user to the group

175. Open `grp-az900-learners` → **Manage** → **Members** → **+ Add members**.
176. Search for ``AZ900 Learner``, select it, click **Select**.
177. Confirm the member count is now 1.

In real deployments you assign roles and licences to **groups**, not individual users - new hires inherit access just by joining the group.

Step 6 - Sign in as the new user

178. Open a **private/incognito** browser window and go to <https://portal.azure.com>.
179. Sign in as `az900.learner@<yourtenant>.onmicrosoft.com` with the auto-generated password.
180. You are forced to **update the password** - set a new one and note it (keep MFA enrolment prompts minimal if offered; you can select "Ask later" where available).
181. The portal opens. **Authentication succeeded** - the tenant verified who this user is.

Step 7 - Observe: authenticated but not authorised

Still in the private window as `az900.learner`:

182. Search **Subscriptions** - the list is empty ("You don't have any subscriptions").
183. Search **Resource groups** - no resource groups are visible, even though `rg-az900-labs` exists in the same tenant.
184. Try **Create a resource** → pick anything → the Basics tab has no subscription to select, so creation is impossible.

This is the key lesson: Entra ID **authenticated** the user (proved identity), but with no Azure **RBAC role assignments** the user is **authorised** to do nothing with resources. Lab 12 fixes this by assigning a role. Sign out of the private window but keep the user and its new password.

Step 8 - Look at authentication methods and MFA (look, don't enforce)

Back in your admin window:

185. In Entra ID, click **Manage** → **Security** → **Authentication methods** (or **Manage** → **Users** → select a user → **Authentication methods**).
186. Browse the available methods: Microsoft Authenticator, SMS, FIDO2 security keys, Temporary Access Pass.

187. ****Do NOT enable any enforcement policy**** - just note that multifactor authentication requires two or more of: something you know (password), something you have (phone/key), something you are (biometric).

Free-tier tenants also have **Security defaults** (Entra ID → Overview → Properties → Manage security defaults), which enforce MFA tenant-wide - leave it as is for this course.

Clean up

Keep the user `az900.learner` and the group `grp-az900-learners` - Lab 12 (RBAC) reuses them. They cost nothing.

If you are not continuing to Lab 12, delete them: Entra ID → **Users** → tick **AZ900 Learner** → **Delete**, and **Groups** → tick **grp-az900-learners** → **Delete**. Keep the resource group **rg-az900-labs** - it is reused until Lab 16.

What you learned

- ****Microsoft Entra ID**** is Azure's cloud identity service; every subscription trusts one tenant, identified by a primary domain and tenant ID.
- ****Users**** are identities; ****security groups**** (Assigned or Dynamic) are the unit you should manage access with.
- ****Authentication**** (proving who you are) is separate from ****authorization**** (what you may do) - a new user can sign in yet see zero subscriptions or resources.
- Azure resource access comes from ****RBAC role assignments****, not from merely existing in the tenant (explored in Lab 12).
- ****MFA**** strengthens authentication by combining something you know, have, and are; Entra ID supports Authenticator, SMS, and FIDO2 keys.

Lab 12 - Manage Access with Azure RBAC

In this lab you explore Azure role-based access control (RBAC) on the **rg-az900-labs** resource group: you inspect built-in roles, assign the **Reader** role to the Lab 11 user **az900.learner**, prove that authorization blocks that user from creating resources, and then remove the assignment. This maps to the AZ-900 skill area "Describe Azure management and governance" (RBAC, scope and least privilege) and reinforces the difference between authentication (Microsoft Entra ID) and authorization (RBAC). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 3 - Describe Azure management and governance | Source: [labs/lab-12-rbac.md](#)

Step 1 - Open Access control (IAM) on the resource group

188. In the portal search bar, type **Resource groups** and open it.
189. Click **rg-az900-labs** (if it does not exist, click **+ Create**, name it `rg-az900-labs`, region **Southeast Asia**, then open it).
190. In the left menu, click **Access control (IAM)**.

Every subscription, resource group and resource has this blade - RBAC works at all of these scopes.

Step 2 - View existing role assignments and scope inheritance

191. On the **Access control (IAM)** blade, click the **Role assignments** tab.
192. Find your own account in the list. Note the **Role** column shows **Owner** and the **Scope** column shows **Subscription (Inherited)**.

Role assignments flow downward: management group → subscription → resource group → resource. You never assigned Owner on this resource group directly - you inherited it from the subscription.

Step 3 - Compare Reader, Contributor and Owner role definitions

193. Click the **Roles** tab.
194. In the search box type **Reader**, then click **View** (under the **Details** column) on the **Reader** row.
195. On the **Permissions** tab note the single action: `*/read`.
196. Click the **JSON** tab to see the raw role definition. Close the pane.
197. Repeat for **Contributor** - actions `/*` but with **NotActions** blocking `Microsoft.Authorization/*Write` (cannot grant access to others).
198. Repeat for **Owner** - actions `/*` with no RBAC restrictions.

| Role | Can view | Can create/delete | Can grant access |

|---|---|---|

| Reader | Yes | No | No |

| Contributor | Yes | Yes | No |

| Owner | Yes | Yes | Yes |

Step 4 - Assign the Reader role to az900.learner

199. Back on **Access control (IAM)**, click **+ Add** > **Add role assignment**.

200. On the **Role** tab, under **Job function roles**, select **Reader** and click **Next**.
201. On the **Members** tab:

- **Assign access to**: `User, group, or service principal`
- Click **+ Select members**, search for `az900.learner`, select the user, click **Select**.

202. Click **Review + assign** twice.

You have granted least-privilege access: the learner can see everything in this resource group but change nothing.

Step 5 - Test the assignment as az900.learner

203. Open a **private/incognito** browser window and go to <https://portal.azure.com>.
204. Sign in as `az900.learner@<yourtenant>.onmicrosoft.com` (password from Lab 11; change it if prompted).
205. Search for **Resource groups** - the user can now see **rg-az900-labs** and open the resources inside it (e.g. the Lab 9 storage account).

Signing in successfully is **authentication** (Entra ID proved who the user is); seeing the resource group is **authorization** (RBAC decided what they may do).

Step 6 - Prove that Reader blocks create and delete

206. Still signed in as `az900.learner`, open **rg-az900-labs** and click **+ Create**.
207. Search for **Storage account**, click **Create**, pick any name and click **Review + create** > **Create**.
208. The deployment fails with an authorization error similar to:

- `The client 'az900.learner@...' with object id '...' does not have authorization to perform action 'Microsoft.Storage/storageAccounts/write' over scope '/subscriptions/.../resourceGroups/rg-az900-labs' ...`

209. Also note that on any existing resource the **Delete** button is greyed out or fails the same way.
210. Sign out and close the private window.

Read the error carefully - it names the exact action and scope that was denied. This is RBAC enforcing least privilege.

Step 7 - Use Check access to query effective permissions

211. Back in your own (Owner) session, open **rg-az900-labs** > **Access control (IAM)** > **Check access** tab.
212. Click **Check access**, search for `az900.learner`, and select the user.
213. The pane lists the user's current assignments at this scope: **Reader**.

Check access is the quickest way to answer "what can this person actually do here?" without reading every assignment.

Step 8 - (Optional) List role assignments with the Azure CLI

Open Cloud Shell (the  icon in the portal top bar, choose **Bash**) and run:

```
az role assignment list --resource-group rg-az900-labs -o table
```

--resource-group scopes the query and -o table gives readable columns; you should see the az900.learner Reader assignment (inherited subscription-level assignments are not shown unless you add --include-inherited).

Step 9 - Remove the role assignment

214. Go to **rg-az900-labs** > **Access control (IAM)** > **Role assignments** tab.
215. Tick the checkbox next to the **Reader** assignment for **az900.learner**.
216. Click **Remove** (or **Delete**), then confirm.

Removing an assignment revokes access immediately - the user keeps their Entra ID account (authentication) but loses authorization at this scope.

Clean up

217. Confirm the Reader assignment for `az900.learner` on `rg-az900-labs` is removed (Step 9).
218. No resources were created in this lab, so nothing else to delete. Keep `rg-az900-labs`, the Lab 11 user and group - later labs still use them.

What you learned

- RBAC assignments combine a **security principal + role definition + scope**, and assignments inherit down from management group → subscription → resource group → resource.
- **Reader**, **Contributor** and **Owner** are the core built-in roles; only Owner can manage access for others.
- **Authentication** (Microsoft Entra ID verifies identity) is separate from **authorization** (RBAC decides permitted actions).
- Assigning the least-privileged role needed (least privilege) limits the impact of mistakes and compromised accounts.
- **Check access** and `az role assignment list` let you audit who can do what at a given scope.

Lab 13 - Apply Resource Tags and Resource Locks

In this lab you organise resources with tags (for ownership, environment and cost reporting) and protect them with resource locks that block accidental deletion - two core Azure governance tools. This maps to the AZ-900 skill area "Describe Azure management and governance" (purpose of tags, and resource locks). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 3 - Describe Azure management and governance | Source: [labs/lab-13-tags-locks.md](#)

Step 1 - Add tags to the resource group

219. Search for **Resource groups**, open **rg-az900-labs** (create it in **Southeast Asia** if missing).
220. In the left menu, click **Tags**.
221. Enter the following name/value pairs, then click **Apply**:

| Name | Value |

|---|---|

| env | training |

| costcenter | az900 |

| owner | <your name> |

Tags are simple metadata - Azure never interprets the values, but Cost Management and inventory reports can group and filter by them.

Step 2 - Confirm tags are NOT inherited by resources

222. Still inside **rg-az900-labs**, open the **Overview** blade and click the Lab 9 storage account (**staz900...**).
223. In its left menu, click **Tags** - the list is **empty**.

By default, tags on a resource group do **not** flow down to the resources inside it. Inheritance can only be forced with an Azure Policy (**Inherit a tag from the resource group**) - remember this distinction for the exam.

Step 3 - Tag an individual resource

224. On the storage account's **Tags** blade, add:

- **Name**: `env`, **Value**: `training`
- **Name**: `costcenter`, **Value**: `az900`

225. Click **Apply**.

A resource can carry up to 50 tags of its own, independent of its resource group's tags.

Step 4 - Filter All resources by tag

226. Search for **All resources** and open it.

227. Click **+ Add filter**, choose **Filter**: ``Tags``, then pick **Tag**: ``env`` and **Value**: ``training``, and click **Apply**.
228. Only the tagged storage account appears (the resource group itself is not a "resource" in this view).

This is how large organisations slice thousands of resources by department, environment or cost centre in seconds.

Step 5 - Manage tags with the Azure CLI

Open Cloud Shell (the `>_` icon, **Bash**) and run:

```
az group update --name rg-az900-labs --set tags.project=az900-course
```

`--set tags.<name>=<value>` adds one tag to the resource group without touching the existing ones; the command echoes the group JSON including the full **tags** block.

```
az tag list --resource-id $(az group show --name rg-az900-labs --query id -o tsv)
```

`az tag list` reads all tags on any resource ID - here the inner `az group show --query id -o tsv` fetches the resource group's full ID. (Note: `az tag create --resource-id <id> --tags ...` also works, but it **replaces** all existing tags, so `az group update --set` is safer for adding one.)

Step 6 - Add a Delete (CanNotDelete) lock on the resource group

229. Open `rg-az900-labs` > left menu **Settings** > **Locks**.
230. Click **+ Add**:

- Lock name**: ``lock-az900-nodelete``
- Lock type**: ``Delete``
- Notes**: ``AZ-900 lab - prevent accidental deletion``

231. Click **OK**.

A lock at resource-group scope is inherited by **every** resource inside the group - unlike tags, locks do flow down.

Step 7 - Try to delete a locked resource

232. Open the storage account ``staz900...`` and click **Delete** on its Overview toolbar; type the account name to confirm.
233. The operation fails with an error similar to:

- ``The scope
'.../resourceGroups/rg-az900-labs/providers/Microsoft.Storage/storageAccounts/staz900...' cannot perform delete operation because following scope(s) are locked:
'/subscriptions/.../resourceGroups/rg-az900-labs'. Please remove the lock and try again.``

Note that this blocked **you, the Owner** - locks apply regardless of RBAC role, which is exactly why they exist: to stop authorised users making accidental changes.

Step 8 - Understand ReadOnly vs Delete (CanNotDelete)

| Lock type | Portal name | Effect |

|---|---|---|

| CanNotDelete | Delete | Authorised users can read and **modify** the resource, but cannot delete it |

| ReadOnly | Read-only | Authorised users can only read - no modify, no delete (can break apps that need write operations, e.g. listing storage keys) |

Use **Delete** locks routinely on production resource groups; use **ReadOnly** sparingly because it can block normal operations.

Step 9 - Remove the lock (required for later clean-up!)

234. Go to **rg-az900-labs** > **Settings** > **Locks**.

235. Click **Delete** next to `lock-az900-nodelete` and confirm.

Or with the CLI:

```
az lock delete --name lock-az900-nodelete --resource-group rg-az900-labs
```

The command returns silently on success. **Do not skip this** - Lab 16's final clean-up deletes the whole resource group, and a leftover lock will block it.

Clean up

236. Ensure the lock `lock-az900-nodelete` is deleted (Step 9).

237. Leave the tags in place - they cost nothing and Lab 15 uses them when grouping costs. Keep `rg-az900-labs` and the storage account for the remaining labs.

What you learned

- Tags are name/value metadata used to organise resources for ownership, environment and cost reporting - they do not affect how a resource runs.
- Resource-group tags are **not inherited** by child resources by default (only an Azure Policy can enforce inheritance).
- Resource locks (**Delete/CanNotDelete** and **Read-only/ReadOnly**) prevent accidental change and apply even to Owners, overriding RBAC permissions.
- Locks at a parent scope (resource group) are inherited by all child resources.
- Governance tooling like tags and locks addresses accidents and organisation - RBAC addresses *who* is allowed to act.

Lab 14 - Enforce Standards with Azure Policy

In this lab you use Azure Policy to enforce an organisational standard - restricting which regions resources may be deployed to - and watch a non-compliant deployment get denied at creation time, even for an Owner. This maps to the AZ-900 skill area "Describe Azure management and governance" (purpose of Azure Policy and compliance). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 3 - Describe Azure management and governance | Source: [labs/lab-14-azure-policy.md](#)

Step 1 - Open the Policy service

238. In the portal search bar, type **Policy** and open the **Policy** service.
239. On the **Overview** blade, note the compliance summary for your subscription (likely 100% or "no assignments" if this is a fresh account).

Azure Policy evaluates every resource against assigned rules and reports compliance continuously - it is governance for *what* gets created, not *who* creates it.

Step 2 - Browse built-in policy definitions

240. In the Policy left menu, under **Authoring**, click **Definitions**.
241. In the **Search** box type **Allowed locations** - open the built-in definition named **Allowed locations**.
242. Read the description and click the **JSON** view: the rule denies any resource whose ``location`` is not in the ``listOfAllowedLocations`` parameter.
243. Go back and search **Require a tag on resources** - open it and note it denies creation of resources missing a given tag.

Azure ships hundreds of built-in definitions (many grouped into **initiatives** - bundles of related policies); you assign them rather than writing JSON from scratch.

Step 3 - Assign "Allowed locations" to rg-az900-labs

244. From the **Allowed locations** definition page, click **Assign policy** (or go to **Assignments** > **Assign policy** and pick the definition).
245. On the **Basics** tab:
 - **Scope**: click the ``...`` button, select your subscription, then **Resource Group**: ``rg-az900-labs``, click **Select**.
 - **Assignment name**: leave as ``Allowed locations``.
246. On the **Parameters** tab:
 - **Allowed locations**: tick **Southeast Asia** and **East Asia** only.
247. Click **Review + create** > **Create**.

The scope matters: this rule now applies only inside **rg-az900-labs**; assigned at subscription or management-group scope it would govern everything below.

Step 4 - Wait for the assignment to take effect

248. Go to **Policy** > **Assignments** and confirm the **Allowed locations** assignment is listed with scope ``rg-az900-labs``.
249. Wait about **5 minutes** before testing - new assignments take a few minutes to propagate to the resource providers that enforce them.

Enforcement (deny at deployment) starts within minutes; the **Compliance** dashboard scan is slower (see Step 6).

Step 5 - Attempt a non-compliant deployment

250. Open `rg-az900-labs` > **Create**, search **Storage account**, click **Create**.
251. On the **Basics** tab:
- Resource group**: ``rg-az900-labs``
 - Storage account name**: ``staz900policytest`` plus random digits (lowercase, unique)
 - Region**: ``West Europe`` ← deliberately not allowed
 - Redundancy**: ``Locally-redundant storage (LRS)``
252. Click **Review + create**. Validation fails (or the deployment is denied on **Create**) with an error like:
- ``Resource 'staz900policytest...' was disallowed by policy. Policy identifiers: [{"policyAssignment":{"name":"Allowed locations", ...}}]`` (Code: RequestDisallowedByPolicy)
253. Read the error - it names the exact **policy assignment** that blocked you. Cancel the deployment (do not retry with an allowed region).

You are the subscription Owner and were still denied: Azure Policy applies to **everyone**, regardless of RBAC role.

Step 6 - View the Compliance blade

254. Go to **Policy** > **Compliance**.
255. Click the **Allowed locations** assignment to see its compliance state and the count of compliant resources (your existing Southeast Asia resources comply).

Note: the compliance evaluation cycle can take **~15 minutes or more** to first populate - if it shows "Not started", continue and check back later. Denied deployments never appear here because the resource was never created.

Step 7 - Compare Azure Policy with RBAC

| | Azure RBAC (Lab 12) | Azure Policy (this lab) |

|---|---|---|

| Question answered | **Who** can perform actions? | **What** configurations are allowed? |

| Applies to | The assigned principal only | Everyone, including Owners |

| Example | Reader cannot create resources | No one can create resources in West Europe |

| Blocked how | Authorization error before the action | **RequestDisallowedByPolicy** on deployment / flagged non-compliant |

Real governance uses both together: RBAC limits who acts, Policy limits what even authorised users can do.

Step 8 - Delete the policy assignment

256. Go to **Policy** > **Assignments**.

257. Click the `...` at the right of the **Allowed locations** assignment and select **Delete assignment**, then confirm.

Or with the CLI in Cloud Shell:

```
az policy assignment list --resource-group rg-az900-labs -o table
az policy assignment delete --name "Allowed locations" --resource-group rg-az900-labs
```

The first command shows the assignment's name at the resource-group scope; the second removes it so later labs can deploy to any region.

Clean up

258. Confirm the **Allowed locations** assignment is deleted (Step 8) - the built-in *definition* stays, only your *assignment* is removed.

259. No resources were created (the test deployment was denied), so nothing else to delete. Keep `rg-az900-labs` for the remaining labs.

What you learned

- Azure Policy enforces organisational standards (allowed regions, required tags, allowed SKUs) and continuously assesses **compliance**.
- A policy is a **definition** (the rule) plus an **assignment** (the rule applied at a scope); initiatives bundle related definitions.
- Policy assignments inherit down the scope hierarchy just like RBAC, but apply to **all users including Owners**.
- Deny-effect policies block non-compliant deployments with `RequestDisallowedByPolicy`, naming the assignment responsible.
- RBAC answers *who can act*; Policy answers *what is allowed* - the AZ-900 exam tests this distinction.

Lab 15 - Estimate and Control Costs: Pricing Calculator, TCO and Budgets

In this lab you estimate the price of a workload with the Azure Pricing Calculator, compare on-premises versus Azure costs with the TCO Calculator, and then control real spend with Cost analysis and a Budget alert in the portal. This maps to the AZ-900 skill area "Describe Azure management and governance" (factors that affect cost, pricing and TCO calculators, Cost Management). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 3 - Describe Azure management and governance | Source: [labs/lab-15-cost-management.md](#)

Step 1 - Part A: Open the Pricing Calculator and add a Virtual Machine

260. In a new browser tab, open <https://azure.microsoft.com/pricing/calculator/> (sign-in optional, but sign in to save estimates).

261. On the **Products** tab, click the **Virtual Machines** tile - it is added to your estimate at the bottom of the page.

262. Scroll down to the **Virtual Machines** section of the estimate and configure:

| Field | Value |

|---|---|

| Region | Southeast Asia |

| Operating system | Windows |

| Type | (OS only) |

| Tier | Standard |

| Instance | B2s: 2 vCPUs, 4 GB RAM |

| Hours | 730 (1 month, 24×7) |

Note the monthly price shown - this is the compute meter only; disks, networking and licensing appear as separate line items.

Step 2 - Add a Storage account to the estimate

263. Scroll back to the top, on the **Products** tab click **Storage Accounts**.

264. In the new **Storage Accounts** section of the estimate set:

- **Region**: `Southeast Asia`
- **Type**: `Block Blob Storage`, **Performance tier**: `Standard`, **Redundancy**: `LRS`
- **Capacity**: `100 GB`

265. Observe the running **Estimated monthly cost** total at the bottom.

Every Azure service is priced per meter (capacity, transactions, egress) - the calculator makes those meters visible before you spend anything.

Step 3 - Compare Pay-as-you-go with savings plans and reservations

266. In the Virtual Machines section, find the **Savings options** area.

267. Switch between:

- **Pay as you go**
- **1 year savings plan** (or **1 year reserved**)
- **3 year reserved**

268. Note how the monthly VM price drops (typically ~30-60% for 1-3 year commitments).

269. Also try changing **Region** to `West US` or `West Europe` and watch the price change, then set it back to `Southeast Asia`.

Key cost factors: resource type and size, consumption (hours/GB), **region**, commitment model, and network egress.

Step 4 - Export or share the estimate

270. At the bottom of the estimate, click **Export** to download the estimate as an Excel file, or

271. Click **Save** then **Share** to generate a link (requires sign-in).

Shared estimates are how architects present costed designs to stakeholders before deployment.

Step 5 - Part B: Model an on-premises workload in the TCO Calculator

272. Open <https://azure.microsoft.com/pricing/tco/calculator/> in a new tab.

273. Under **Define your workloads** > **Servers**, click **+ Add server workload**:

- **Name**: `On-prem web servers`
- **Workload**: `Windows/Linux Server`, **Environment**: `Virtual machines`
- **Operating system**: `Windows`, **VMs**: `2`, **Cores per VM**: `2`, **RAM per VM**: `8` GB

274. Under **Storage**, click **+ Add storage**:

- **Name**: `On-prem SAN`, **Storage type**: `Local Disk/SAN`, **Disk type**: `HDD`, **Capacity**: `1` TB

275. Leave **Networking** at defaults and click **Next**.

The TCO Calculator captures costs the pricing calculator does not: hardware, electricity, data-centre space, and IT labour.

Step 6 - View the 5-year TCO comparison report

276. On the **Adjust assumptions** page, review the defaults (electricity price, IT labour cost, virtualisation ratio) and click **Next**.

277. On the **View report** page, set **Timeframe**: `5 years` and **Region**: `Southeast Asia`.

278. Review the chart comparing **total cost of running on-premises** vs **cost of the equivalent in Azure**, and the per-category breakdown (compute, data centre, networking, storage, IT labour).

279. Optionally click **Download** to save the report as PDF.

TCO = total cost of ownership over time, not just the sticker price - this is the business case tool, while the Pricing Calculator is the engineering estimate tool.

Step 7 - Part C: Explore Cost analysis in the portal

280. Back in <https://portal.azure.com>, search **Cost Management + Billing** and open it.
281. Click **Cost Management** > **Cost analysis** (scope: your subscription).
282. Review the accumulated cost chart for the current month.
283. Change **Group by** to `Resource` to see which resources are generating cost; also try `Tag` > `costcenter` to see your Lab 13 tags at work.

Note: on a free account with no charges yet, Cost analysis may show \$0 or "no data" - the tooling only lights up once billable usage exists. That is expected.

Step 8 - Create a budget with an email alert

284. In **Cost Management**, click **Budgets** > **+ Add**.
285. On the **Create budget** page:

- **Scope**: your subscription
- **Name**: `budget-az900`
- **Reset period**: `Billing month`
- **Amount (\$)**: `20`

286. Click **Next** to the **Set alerts** tab:

- **Alert conditions** - **Type**: `Actual`, **% of budget**: `80` (i.e. alert at \$16 actual spend)
- **Alert recipients (email)**: your email address

287. Click **Create**.

Budgets do **not** stop spending by themselves - they notify you (and can trigger action groups) so you act before costs run away.

Step 9 - (Optional) Review budgets and costs with the CLI

Open Cloud Shell (**Bash**) and run:

```
az consumption budget list -o table
```

Lists your budgets including `budget-az900` with its amount and time grain (if the command prompts, allow it to install the `consumption` extension).

Clean up

288. In **Cost Management** > **Budgets**, you may keep `budget-az900` (it is free and useful) - or select it, click the `...` menu and **Delete** it now.
289. The Pricing Calculator and TCO Calculator estimates live outside your subscription and cost nothing; delete saved estimates from the calculator site if you wish.
290. Keep `rg-az900-labs` and its resources - Lab 16 performs the final course clean-up.

What you learned

- The **Pricing Calculator** estimates the monthly cost of Azure services before deployment; the **TCO Calculator** compares full on-premises ownership costs (hardware, power, labour) against Azure over multiple years.
- Cost is driven by resource type/size, consumption, **region**, commitment model (pay-as-you-go vs savings plan vs reservation) and network egress.

- Reservations and savings plans trade a 1-3 year commitment for significant discounts on steady workloads.
- ****Cost analysis**** breaks real spend down by resource, service or tag - tags applied in Lab 13 become cost-reporting dimensions.
- ****Budgets**** with alert thresholds notify you at defined spend levels but do not block spending on their own.

Lab 16 - Monitor Your Environment: Advisor, Service Health and Azure Monitor

In this final lab you tour Azure's monitoring stack: Azure Advisor's recommendation categories, Service Health alerts for platform incidents, and Azure Monitor's Metrics and Activity log - then you perform the full end-of-course clean-up. This maps to the AZ-900 skill area "Describe Azure management and governance" (Azure Advisor, Service Health, Azure Monitor including Log Analytics and alerts). Run on <https://portal.azure.com> - sign in with your Azure free account (<https://azure.microsoft.com/free>).

Note: Day 2 | Domain 3 - Describe Azure management and governance | Source: [labs/lab-16-monitoring.md](#)

Step 1 - Review Azure Advisor recommendations

291. In the portal search bar, type **Advisor** and open **Azure Advisor**.
292. On the **Overview** blade, note the five recommendation categories, each with a score/count:
 - **Cost** - e.g. shut down or resize under-used VMs
 - **Security** - findings sourced from Microsoft Defender for Cloud
 - **Reliability** - e.g. enable backups, add redundancy
 - **Operational excellence** - e.g. use resource locks, follow deployment best practice
 - **Performance** - e.g. upgrade SKUs or storage tiers for speed
293. Click into one category (e.g. **Cost**) and open any recommendation to see the affected resource and suggested action.

A small lab subscription may show few or zero recommendations - Advisor only advises on what you actually run. Memorise the five categories for the exam.

Step 2 - Download the Advisor recommendations CSV

294. On the Advisor **Overview** (or any category blade), click **Download as CSV** in the toolbar.
295. Open the file and note the columns: category, impact (High/Medium/Low), recommendation and impacted resource.

Teams feed this export into review meetings - Advisor is free, personalised guidance, not a billed service.

Step 3 - Explore Service Health

296. Search **Service Health** and open it.
297. Walk through the left-menu views:
 - **Service issues** - active Azure platform outages affecting *your* subscriptions/regions (usually "No active events" - good news)
 - **Planned maintenance** - upcoming Microsoft maintenance windows
 - **Health advisories** - changes needing action, e.g. service retirements
 - **Health history** - past events
298. Compare with **Azure status** (<https://azure.status.microsoft>) - that page shows global platform health for everyone; Service Health is *personalised* to your resources.

Service Health tells you when the problem is Azure's, not yours - key exam distinction versus Azure Monitor (your resources) and Resource Health (a single resource).

Step 4 - Create an action group for notifications

299. In **Service Health**, click **Health alerts** > **+ Add service health alert** (this opens the alert rule creation wizard; you will create the action group inside it).

300. On the **Condition** tab set:

- **Subscription**: your subscription
- **Services**: `Select all`
- **Regions**: untick all, tick **Southeast Asia** only
- **Event types**: `Select all` (Service issue, Planned maintenance, Health advisories, Security advisory)

301. Go to the **Actions** tab and click **+ Create action group**:

- **Resource group**: `rg-az900-labs`
- **Action group name**: `ag-az900-email`
- **Display name**: `az900email`
- On the **Notifications** tab: **Notification type**: `Email/SMS message/Push/Voice`, **Name**: `email-me`, tick **Email** and enter your email address, click **OK**.
- Click **Review + create** > **Create**.

An action group is a reusable bundle of "who to notify / what to run" - one group can serve many alert rules.

Step 5 - Finish creating the Service Health alert rule

302. Back on the alert wizard's **Actions** tab, confirm `ag-az900-email` is selected.

303. On the **Details** tab:

- **Resource group**: `rg-az900-labs`
- **Alert rule name**: `alert-servicehealth-sea`

304. Click **Review + create** > **Create**.

305. Check your inbox - Azure sends a confirmation email that you were added to the action group.

From now on, any Azure incident touching Southeast Asia would email you automatically instead of you discovering it from angry users.

Step 6 - Chart a metric in Azure Monitor

306. Search **Monitor** and open **Azure Monitor**, then click **Metrics**.

307. In the **Select a scope** pane, expand your subscription > `rg-az900-labs` and select a resource:

- If a VM still exists from earlier labs: select it, **Metric**: `Percentage CPU`, **Aggregation**: `Avg`.
- Otherwise select the Lab 9 storage account `staz900...`, **Metric namespace**: `Account`, **Metric**: `Transactions`, **Aggregation**: `Sum`.

308. Change the time range (top right) to **Last 24 hours** and watch the chart update.

309. Click **Save to dashboard** > **Pin to dashboard** > **Create new** > name it `AZ900 Ops` > **Pin**.

310. Open **Dashboard** from the portal menu to see your pinned chart.

Metrics are numeric time-series collected automatically for every resource - no agent or setup required for platform metrics.

Step 7 - Audit operations with the Activity log

311. In **Azure Monitor**, click **Activity log** (also available on every resource and resource group).

312. Set the filters: **Timespan**: `Last 24 hours`, and add filter **Resource group**: `rg-az900-labs`.

313. Find entries from earlier labs - role assignments (Lab 12), lock create/delete (Lab 13), policy assignment (Lab 14). Click one and open the **JSON** tab to see **caller** (who), **operationName** (what) and timestamp (when).

The Activity log is the subscription-level audit trail of control-plane operations - "who did what, when" - retained for 90 days by default.

Step 8 - Understand Log Analytics and KQL (concept only)

No workspace needed for this course - just know the model:

- **Log Analytics workspace**: a store into which resources, VMs and services send detailed logs.
- **KQL (Kusto Query Language)**: the query language used across Log Analytics, e.g. ``AzureActivity | where OperationNameValue contains "delete" | take 10``.
- Alert rules can fire on log query results as well as on metrics.

For AZ-900 you only need to identify Azure Monitor's parts: Metrics, Logs (Log Analytics + KQL), Alerts (with action groups), and visualisation (dashboards/workbooks).

Step 9 - FINAL COURSE CLEAN-UP: remove alert rule and action group

314. Go to **Azure Monitor** > **Alerts** > **Alert rules**, tick `alert-servicehealth-sea`, click **Delete**, confirm.

315. Go to **Azure Monitor** > **Alerts** > **Action groups** (or search **Action groups**), tick `ag-az900-email`, click **Delete**, confirm.

Delete the alert rule before the action group - a rule referencing a deleted group would silently notify no one.

Step 10 - FINAL COURSE CLEAN-UP: delete the resource group

First check for the Lab 13 lock - a leftover lock will block deletion:

```
az lock list --resource-group rg-az900-labs -o table
az lock delete --name lock-az900-nodelock --resource-group rg-az900-labs
```

Run the second command only if the first shows the lock still exists. Then delete everything the course created in one shot:

```
az group delete --name rg-az900-labs --yes --no-wait
```

--yes skips the confirmation prompt and --no-wait returns immediately while Azure deletes the group and every resource inside it (storage account, dashboards' source resources, action-group container) in the background.

```
az group exists --name rg-az900-labs
```

Re-run this after a few minutes until it prints **false**.

Step 11 - FINAL COURSE CLEAN-UP: delete the Entra ID test identities

316. Search **Microsoft Entra ID** and open it.
317. Click **Users**, tick `az900.learner`, click **Delete**, confirm. (The user moves to **Deleted users** for 30 days, then purges automatically.)
318. Click **Groups**, tick `grp-az900-learners`, click **Delete**, confirm.

Or with the CLI:

```
az ad user delete --id az900.learner@<yourtenant>.onmicrosoft.com
az ad group delete --group grp-az900-learners
```

Replace **<yourtenant>** with your tenant name; both commands return silently on success.

Step 12 - Verify the subscription is empty

319. Open **All resources** - the list should be empty (allow a few minutes for the group deletion to finish).
320. Open **Resource groups** - `rg-az900-labs` should be gone.
321. Optionally open **Cost Management** > **Cost analysis** once more to confirm no ongoing charges.

An empty subscription means zero ongoing cost - the golden rule of lab hygiene. Congratulations, you have completed all 16 AZ-900 labs.

Clean up

All course clean-up was performed in Steps 9-12:

322. Deleted Service Health alert rule `alert-servicehealth-sea` and action group `ag-az900-email`.
323. Removed any remaining resource lock, then deleted resource group `rg-az900-labs` (`az group delete --name rg-az900-labs --yes --no-wait`).
324. Deleted Entra ID user `az900.learner` and group `grp-az900-learners` (Lab 11).
325. Verified **All resources** is empty. If you created the `budget-az900` budget in Lab 15 and want it gone too, delete it under **Cost Management** > **Budgets**.

What you learned

- **Azure Advisor** gives free, personalised recommendations in five categories: Cost, Security, Reliability, Operational excellence and Performance.
- **Service Health** reports Azure platform issues, planned maintenance and health advisories personalised to your subscriptions, and can alert you via action groups.
- **Azure Monitor Metrics** charts platform time-series data with no setup; the **Activity log** is the subscription audit trail of who did what.
- **Log Analytics** stores detailed logs queried with **KQL**, and alert rules plus **action groups** turn signals into notifications and automation.

- Deleting a resource group removes every resource inside it - the fastest way to guarantee a lab environment stops costing money.